

Universidade Federal de Alfenas

Instituto de Ciências Exatas

Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Sistemas de Equações Lineares: Métodos Iterativos em Python

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Modalidade: Atividade de investigação em dupla

Entrega: Relatório .pdf + código-fonte .py

Apresentação

No trabalho anterior, investigamos os métodos *diretos*, que resolvem $Ax = b$ em número finito de passos. Neste plano, o foco se volta para os **métodos iterativos**: Gauss-Jacobi, Gauss-Seidel, SOR e Gradiente Conjugado. Esses métodos geram uma sequência de aproximações que (espera-se) converge para a solução, e são a base da solução de sistemas de grande escala em engenharia e ciência.

O código de cada método é fornecido. Cabe ao aluno *executar, perturbar, medir, comparar e interpretar* — com ênfase na análise das **diferenças** entre os métodos e na compreensão das condições de convergência.

Objetivos

Ao concluir esta investigação, o aluno será capaz de:

- a) Implementar e aplicar os métodos de Gauss-Jacobi e Gauss-Seidel;
- b) Compreender e explorar o método SOR e seu parâmetro ótimo ω ;
- c) Aplicar o Gradiente Conjugado (CG) e o PCG a sistemas esparsos;
- d) Verificar e analisar critérios de convergência (diagonal dominância, raio espectral, Sassenfeld);
- e) Comparar empiricamente a taxa de convergência dos métodos;
- f) Tomar decisões fundamentadas sobre qual método iterativo usar em cada situação.

Atenção

O objetivo **não** é apenas obter a solução correta. É *entender por que* um método converge mais rápido, quando ele falha, e o que o raio espectral tem a ver com tudo isso. Documente todas as observações, inclusive as inesperadas.

Sumário

Apresentação	1
1 Gauss-Jacobi e Gauss-Seidel: Primeiros Passos	3
1.1 Verificação básica e critério de convergência	4
1.2 Quando Jacobi paralela Gauss-Seidel, e quando divergem	5
2 Critérios de Convergência: Teoria vs. Prática	6
2.1 Diagonal dominância e Sassenfeld	7
2.2 Condição necessária vs. suficiente	7
3 Método SOR: O Papel do Parâmetro ω	8
3.1 Sensibilidade ao parâmetro ω	9
3.2 Sobre-relaxação: mecanismo e limites	9
4 Gradiente Conjugado: Convergência por Subespaços	11
4.1 CG vs. SOR: convergência em sistemas SPD	12
4.2 O impacto do número de condição	13
4.3 Pré-condicionamento na prática	13
5 Custo Computacional e Escalabilidade	14
5.1 Como k varia com n ?	14
5.2 Tempo real de execução	15
6 Comparação Global: Quando Usar Cada Método?	16
6.1 Análise comparativa estruturada	16
7 Projeto Integrador: Equação de Calor Estacionária	17
8 Desafio (Opcional — Pontuação Extra)	18
Entrega e Critérios de Avaliação	19
A Esqueleto do Script Principal	20
B Referências Bibliográficas	21

1. Gauss-Jacobi e Gauss-Seidel: Primeiros Passos

Dica

Copie os dois blocos abaixo para `jacobi_seidel.py`. Não os modifique antes de responder as questões desta seção.

```
1 import numpy as np
2
3 def jacobi(A, b, x0=None, tol=1e-8, max_iter=500):
4     """
5     Resolve  $Ax = b$  pelo metodo de Gauss-Jacobi.
6     Retorna (solucao, num_iteracoes, historico_residuos).
7     """
8     A = np.array(A, dtype=float)
9     b = np.array(b, dtype=float)
10    n = len(b)
11    x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
12
13    D_inv = 1.0 / np.diag(A)
14    R = A - np.diag(np.diag(A))
15    historico = []
16
17    for k in range(max_iter):
18        x_new = D_inv * (b - R @ x)
19        residuo = np.linalg.norm(A @ x_new - b)
20        historico.append(residuo)
21        diff = np.linalg.norm(x_new - x, np.inf)
22        denom = np.linalg.norm(x_new, np.inf)
23        if denom > 0 and diff / denom < tol:
24            return x_new, k + 1, historico
25        x = x_new
26
27    return x, max_iter, historico
28
29
30 def gauss_seidel(A, b, x0=None, tol=1e-8, max_iter=500):
31     """
32     Resolve  $Ax = b$  pelo metodo de Gauss-Seidel.
33     Retorna (solucao, num_iteracoes, historico_residuos).
34     """
35     A = np.array(A, dtype=float)
36     b = np.array(b, dtype=float)
37     n = len(b)
38     x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
39     historico = []
40
41     for k in range(max_iter):
42         x_old = x.copy()
43         for i in range(n):
44             soma = (np.dot(A[i, :i], x[:i]))
```

```

45         + np.dot(A[i, i+1:], x_old[i+1:]))
46         x[i] = (b[i] - soma) / A[i, i]
47         residuo = np.linalg.norm(A @ x - b)
48         historico.append(residuo)
49         diff = np.linalg.norm(x - x_old, np.inf)
50         denom = np.linalg.norm(x, np.inf)
51         if denom > 0 and diff / denom < tol:
52             return x, k + 1, historico
53
54     return x, max_iter, historico
55
56
57 def raio_espectral(A, metodo='jacobi'):
58     """Calcula rho(T_J) ou rho(T_GS) numericamente."""
59     D = np.diag(np.diag(A))
60     L = -np.tril(A, -1)
61     U = -np.triu(A, 1)
62     if metodo == 'jacobi':
63         T = np.linalg.inv(D) @ (L + U)
64     else:
65         T = np.linalg.inv(D - L) @ U
66     return np.max(np.abs(np.linalg.eigvals(T))), T

```

Listing 1: jacobi_seidel.py — Gauss-Jacobi e Gauss-Seidel

1.1. Verificação básica e critério de convergência

Q1.1. Execute os dois métodos para o sistema:

$$\begin{pmatrix} 8 & -1 & 1 \\ 1 & 6 & -2 \\ -1 & 2 & 7 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 14 \\ 10 \\ -5 \end{pmatrix}, \quad x^{(0)} = (0, 0, 0)^T, \quad \varepsilon = 10^{-6}.$$

- Verifique que A é estritamente diagonal dominante calculando $\alpha_i = \sum_{j \neq i} |a_{ij}| / |a_{ii}|$ para cada linha.
- Preencha a tabela com os valores obtidos nas primeiras 5 iterações de cada método (imprima $x^{(k)}$ e $\|Ax^{(k)} - b\|_2$ a cada passo):

k	Gauss-Jacobi				Gauss-Seidel			
	x_1	x_2	x_3	$\ r\ $	x_1	x_2	x_3	$\ r\ $
1								
2								
3								
4								
5								

Qual método chegou mais perto da solução exata $x^* = (2; 1; -1)^T$ nas primeiras iterações?

Q1.2. Use `raio_espectral` para calcular $\rho(T_J)$ e $\rho(T_{GS})$ para a matriz do sistema anterior.

- (a) Ambos são menores que 1? O que isso garante?
- (b) Calcule a razão $\rho(T_{GS})/\rho(T_J)$. O que ela indica sobre a velocidade relativa dos dois métodos?
- (c) A estimativa teórica $\|e^{(k)}\| \leq \rho^k \|e^{(0)}\|$ é consistente com os resíduos observados na questão anterior?

1.2. Quando Jacobi paralela Gauss-Seidel, e quando divergem

Q1.3. Ambos os métodos usam a decomposição $A = D - L - U$, mas diferem em *quando* os novos valores são aproveitados.

- (a) Modifique `gauss_seidel` para imprimir, a cada iteração, qual componente usa valor “novo” e qual usa valor “antigo”. Execute para $n = 3$ e confirme a diferença na primeira iteração.
- (b) Gauss-Jacobi pode ser naturalmente paralelizado: cada $x_i^{(k+1)}$ pode ser calculado independentemente. Por que Gauss-Seidel *não* pode? O que impede?
- (c) Construa uma matriz 3×3 diagonal dominante em que Gauss-Seidel usa todos os valores novos no cálculo de $x_3^{(k+1)}$. Há ganho real?

Q1.4. Nem todo sistema converge com Jacobi e Gauss-Seidel. Teste os dois métodos no sistema abaixo (limite de 30 iterações):

$$\begin{pmatrix} 1 & 4 & -1 \\ 3 & 1 & 2 \\ 2 & -1 & 1 \end{pmatrix} x = \begin{pmatrix} 3 \\ 6 \\ 1 \end{pmatrix}.$$

- (a) Calcule $\rho(T_J)$ e $\rho(T_{GS})$. Qual deles é > 1 ?
- (b) O que acontece empiricamente com o resíduo ao longo das iterações? Plote $\|r^{(k)}\|$ vs. k para os dois métodos.
- (c) Reordene as equações para obter diagonal dominância. O método converge após a reordenação? Reporte os novos ρ .

2. Critérios de Convergência: Teoria vs. Prática

Por que estudar critérios de convergência?

A diagonal dominância e o critério de Sassenfeld fornecem *garantias* antes de executar qualquer iteração. Mas essas condições são suficientes, não necessárias: existem matrizes que convergem sem satisfazê-las. Esta parte investiga esse espaço entre teoria e prática.

```

1 import numpy as np
2
3 def criterio_linhas(A):
4     """
5     Verifica diagonal dominancia estrita por linhas.
6     Retorna (satisfaz, alfas) onde alfas[i] = soma_j!=i |a_ij| /
7         |a_ii|.
8     """
9     n = A.shape[0]
10    alfas = np.array([
11        sum(abs(A[i, j]) for j in range(n) if j != i) / abs(A[i,
12        i])
13        for i in range(n)
14    ])
15    return np.all(alfas < 1), alfas
16
17 def criterio_sassenfeld(A):
18     """
19     Criterio de Sassenfeld para Gauss-Seidel.
20     Retorna (satisfaz, betas).
21     """
22    n = A.shape[0]
23    betas = np.zeros(n)
24    betas[0] = sum(abs(A[0, j]) for j in range(1, n)) / abs(A[0,
25    0])
26    for i in range(1, n):
27        soma = sum(abs(A[i, s]) * betas[s] for s in range(i))
28        soma += sum(abs(A[i, s]) for s in range(i + 1, n))
29        betas[i] = soma / abs(A[i, i])
30    return np.max(betas) < 1, betas
31
32 def diagnostico(A, nome="A"):
33     """Imprime diagnostico completo de convergencia para a
34     matriz A."""
35    print(f"\n=== Diagnostico: {nome} ===")
36    ok_lin, alfas = criterio_linhas(A)
37    ok_sas, betas = criterio_sassenfeld(A)
38    print(f"    Criterio das linhas (Jacobi/GS): {'OK' if ok_lin
39        else 'FALHA'} "
40        f" | alpha = {np.max(alfas):.4f}")

```

```

38     print(f"    Criterio de Sassenfeld      (GS): {'OK' if ok_sas
          else 'FALHA'} "
39         f" | beta = {np.max(betas):.4f}")
40     from jacobi_seidel import raio_espectral
41     rho_j, _ = raio_espectral(A, 'jacobi')
42     rho_gs, _ = raio_espectral(A, 'seidel')
43     print(f"    Raio espectral Jacobi:      rho_J = {rho_j:.6f}")
44     print(f"    Raio espectral GS:          rho_GS = {rho_gs:.6f}")
45     print(f"    Convergencia garantida: J={'Sim' if rho_j < 1
          else 'NAO'} "
46         f" | GS={'Sim' if rho_gs < 1 else 'NAO'}")

```

Listing 2: convergencia.py — critérios e diagnósticos

2.1. Diagonal dominância e Sassenfeld

Q2.1. Execute `diagnostico` nas quatro matrizes abaixo. Antes de executar, **preveja** para cada uma qual critério será satisfeito e qual método convergirá:

$$B_1 = \begin{pmatrix} 9 & -1 & 2 \\ -1 & 8 & -1 \\ 2 & -1 & 9 \end{pmatrix}, \quad B_2 = \begin{pmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix},$$

$$B_3 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \quad B_4 = \begin{pmatrix} 5 & -2 & 1 & 0 \\ -2 & 6 & -3 & 1 \\ 1 & -3 & 7 & -2 \\ 0 & 1 & -2 & 5 \end{pmatrix}.$$

Preencha a tabela e depois compare com os resultados reais:

Matriz	Diag. Dom.	Sassenfeld	$\rho(T_J)$	$\rho(T_{GS})$	Jacobi conv.?
B_1					
B_2					
B_3					
B_4					

Qual critério se mostrou mais restritivo? Houve casos em que um método convergiu sem que nenhum critério fosse satisfeito?

2.2. Condição necessária vs. suficiente

Q2.2. Diagonal dominância é condição *suficiente*, não necessária. Construa um exemplo de matriz 3×3 que:

- Não** é diagonal dominante, mas $\rho(T_J) < 1$ (Jacobi converge);
- Não** satisfaz o critério de Sassenfeld, mas $\rho(T_{GS}) < 1$ (GS converge).

Verifique numericamente e explique intuitivamente por que a convergência ocorre mesmo sem a garantia do critério.

Q2.3. Sabemos que para certas classes de matrizes tridiagonais (“Property A”), vale a relação $\rho(T_{GS}) = \rho(T_J)^2$.

- Verifique essa relação numericamente para a matriz B_4 acima.
- Se $\rho(T_J) = 0,8$, quantas iterações de Jacobi e de Gauss-Seidel seriam necessárias para reduzir o erro por um fator 10^{-6} ? (Use $\|e^{(k)}\| \leq \rho^k \|e^{(0)}\|$.)
- Essa relação $\rho_{GS} = \rho_J^2$ significa que GS sempre converge em metade das iterações de Jacobi? Discuta.

3. Método SOR: O Papel do Parâmetro ω

A grande questão do SOR

O SOR generaliza o Gauss-Seidel com um único parâmetro ω . Escolher ω errado pode ser *pior* que Gauss-Seidel — ou até fazer o método divergir. Mas com ω certo, o ganho pode ser de uma ordem de grandeza. Esta parte investiga essa sensibilidade.

```

1 import numpy as np
2
3 def sor(A, b, omega, x0=None, tol=1e-8, max_iter=500):
4     """
5     Resolve Ax = b pelo metodo SOR.
6     omega = 1.0 -> Gauss-Seidel puro.
7     Retorna (solucao, num_iteracoes, historico_residuos).
8     """
9     if not (0 < omega < 2):
10         raise ValueError("omega deve estar em (0, 2)")
11     A = np.array(A, dtype=float)
12     b = np.array(b, dtype=float)
13     n = len(b)
14     x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
15     historico = []
16
17     for k in range(max_iter):
18         x_old = x.copy()
19         for i in range(n):
20             soma = (np.dot(A[i, :i], x[:i])
21                    + np.dot(A[i, i+1:], x_old[i+1:]))
22             x_gs = (b[i] - soma) / A[i, i]
23             x[i] = (1 - omega) * x_old[i] + omega * x_gs
24         residuo = np.linalg.norm(A @ x - b)
25         historico.append(residuo)
26         diff = np.linalg.norm(x - x_old, np.inf)
27         denom = np.linalg.norm(x, np.inf)
28         if denom > 0 and diff / denom < tol:
29             return x, k + 1, historico
30
31     return x, max_iter, historico
32
33
34 def omega_young(A):
35     """
36     Calcula omega_opt pela formula de Young (valida para
37     Property A).
38     Retorna (omega_opt, rho_J, rho_SOR).
39     """
40     D_inv = np.diag(1.0 / np.diag(A))
41     R = A - np.diag(np.diag(A))
42     T_J = D_inv @ R
43     rho_J = np.max(np.abs(np.linalg.eigvals(T_J)))

```

```

43     omega_opt = 2.0 / (1 + np.sqrt(1 - rho_J**2))
44     rho_SOR = omega_opt - 1
45     return omega_opt, rho_J, rho_SOR
46
47
48 def varredura_omega(A, b, omegas, tol=1e-8, max_iter=500):
49     """Executa SOR para cada omega e retorna lista de iteracoes.
50         """
51     resultados = []
52     for w in omegas:
53         try:
54             _, iters, _ = sor(A, b, w, tol=tol, max_iter=
55                             max_iter)
56         except Exception:
57             iters = max_iter
58         resultados.append(iters)
59     return resultados

```

Listing 3: sor.py — SOR e busca do ω ótimo

3.1. Sensibilidade ao parâmetro ω

Q3.1. Considere a matriz tridiagonal 10×10 com diagonal principal 4 e diagonais adjacentes -1 , $b = (3, 3, \dots, 3)^T$.

- Execute `varredura_omega` com $\omega \in \{0, 3; 0, 5; 0, 7; 0, 9; 1, 0; 1, 1; 1, 2; 1, 3; 1, 5; 1, 7; 1, 9\}$. Plote “número de iterações vs. ω ” com ponto vermelho em $\omega = 1, 0$ (Gauss-Seidel). Descreva o formato da curva.
- Calcule ω_{opt} pela fórmula de Young e sobreponha ao gráfico. A curva tem mínimo no ω_{opt} teórico?
- Preencha a tabela comparativa (tolerância 10^{-8}):

Método	ω	Iterações	ρ
Gauss-Jacobi	—		
Gauss-Seidel	1, 0		
SOR (sub)	0, 5		
SOR (ω_{opt})			
SOR (super alto)	1, 9		

3.2. Sobre-relaxação: mecanismo e limites

Q3.2. Com $\omega = 1, 2$, execute SOR no mesmo sistema tridiagonal e imprima $x^{(1)}$ componente a componente. Compare com o $x^{(1)}$ de Gauss-Seidel.

- O SOR “ultrapassou” a solução em alguma componente? Isso é esperado?
- Para $\omega = 1, 95$, o método converge? Plote $\|r^{(k)}\|$ vs. k . O que acontece com o resíduo?
- A condição $0 < \omega < 2$ é necessária para convergência. Teste $\omega = 2, 0$ e $\omega = 2, 1$. O que ocorre?

Q3.3. A fórmula de Young vale para matrizes com “Property A”. a matriz B_2 da Seção 2 (que *não* é tridiagonal pura):

- (a) Calcule ω_{opt} pela fórmula de Young e compare com o ω experimental que minimiza as iterações (via `varredura_omega`).
- (b) A fórmula de Young foi precisa? Discuta as limitações do uso da fórmula fora das condições teóricas.

4. Gradiente Conjugado: Convergência por Subespaços

O que torna o CG especial?

Jacobi, Gauss-Seidel e SOR funcionam para qualquer sistema convergente. O Gradiente Conjugado exige que A seja SPD, mas em troca oferece convergência em no máximo n passos e custo por iteração de $O(\text{nnz})$. Esta parte compara o CG com os métodos anteriores e explora o impacto do condicionamento — e do pré-condicionamento — na prática.

```

1 import numpy as np
2
3 def cg(A, b, x0=None, tol=1e-8, max_iter=1000):
4     """
5     Gradiente Conjugado para A SPD.
6     Retorna (solucao, num_iteracoes, historico_residuos).
7     """
8     A = np.array(A, dtype=float)
9     b = np.array(b, dtype=float)
10    n = len(b)
11    x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
12    r = b - A @ x
13    p = r.copy()
14    rr = np.dot(r, r)
15    historico = [np.sqrt(rr)]
16
17    for k in range(max_iter):
18        Ap = A @ p
19        alpha = rr / np.dot(p, Ap)
20        x = x + alpha * p
21        r = r - alpha * Ap
22        rr_new = np.dot(r, r)
23        historico.append(np.sqrt(rr_new))
24        if np.sqrt(rr_new) < tol:
25            return x, k + 1, historico
26        beta = rr_new / rr
27        p = r + beta * p
28        rr = rr_new
29
30    return x, max_iter, historico
31
32
33 def pcg(A, b, M_inv=None, x0=None, tol=1e-8, max_iter=1000):
34     """
35     Gradiente Conjugado Pre-Condicionado.
36     M_inv: funcao que aplica M^{-1} a um vetor (pre-
37           condicionador).
38     Se M_inv=None, usa M = I (equivalente ao CG puro).
39     Retorna (solucao, num_iteracoes, historico_residuos).
40     """

```

```

40     if M_inv is None:
41         M_inv = lambda v: v.copy()
42     A = np.array(A, dtype=float)
43     b = np.array(b, dtype=float)
44     n = len(b)
45     x = np.zeros(n) if x0 is None else np.array(x0, dtype=float)
46     r = b - A @ x
47     z = M_inv(r)
48     p = z.copy()
49     rz = np.dot(r, z)
50     historico = [np.linalg.norm(r)]
51
52     for k in range(max_iter):
53         Ap = A @ p
54         alpha = rz / np.dot(p, Ap)
55         x = x + alpha * p
56         r = r - alpha * Ap
57         historico.append(np.linalg.norm(r))
58         if np.linalg.norm(r) < tol:
59             return x, k + 1, historico
60         z = M_inv(r)
61         rz_new = np.dot(r, z)
62         beta = rz_new / rz
63         p = z + beta * p
64         rz = rz_new
65
66     return x, max_iter, historico

```

Listing 4: gradiente_conjugado.py — CG e PCG

4.1. CG vs. SOR: convergência em sistemas SPD

Q4.1. Use a matriz tridiagonal $n \times n$ ($b_i = 4$, $a_i = c_i = -1$) para $n = 20$ como campo de teste.

- (a) Execute Gauss-Jacobi, Gauss-Seidel, $\text{SOR}(\omega_{\text{opt}})$ e CG com tolerância 10^{-8} . Preencha:

Método	Iterações	$\ r_{\text{final}}\ $	Custo/iter.	Tipo
Gauss-Jacobi			$O(n^2)$	estacionário
Gauss-Seidel			$O(n^2)$	estacionário
SOR (ω_{opt})			$O(n^2)$	estacionário
Gradiente Conj.			$O(\text{nnz})$	Krylov

- (b) Plote as curvas de convergência $\|r^{(k)}\|$ vs. k para os quatro métodos em escala logarítmica no mesmo gráfico.
- (c) O CG converge em no máximo $n = 20$ iterações? Em quantas ele efetivamente convergiu?

4.2. O impacto do número de condição

Q4.2. Gere matrizes SPD com número de condição controlado: $A = Q @ np.diag(lambdas) @ Q.T$, onde Q é uma matriz ortogonal aleatória (`scipy.stats.ortho_group`) e $lambdas$ vai de 1 a κ . Use $n = 30$ e $\kappa \in \{5, 50, 500, 5000\}$.

- Para cada κ , execute CG (tolerância 10^{-8}) e registre o número de iterações.
- Plote “iterações vs. κ ” em escala log-log.
- A estimativa teórica $k \approx \frac{1}{2}\sqrt{\kappa} \ln(2/\varepsilon)$ é compatível com os dados?
- Agora compare com Gauss-Seidel para $\kappa = 5000$. Qual método é mais afetado pelo mau condicionamento?

4.3. Pré-condicionamento na prática

Q4.3. Implemente dois pré-condicionadores simples e compare seu efeito. Para a matriz do item anterior com $\kappa = 5000$:

- Pré-cond. Jacobi:** $M = \text{diag}(A)$, $M^{-1}v = v/\text{diag}(A)$. Use `pcg` com esse pré-condicionador.
- Calcule $\kappa(M^{-1}A)$ e compare com $\kappa(A)$. O condicionamento melhorou?
- Preencha a tabela:

Configuração	Iterações	κ efetivo	$\ r_{final}\ $
CG sem pré-cond.		≈ 5000	
PCG + Jacobi			

- Por que o pré-condicionamento Jacobi é eficaz quando a diagonal domina, e ineficaz quando A tem entradas diagonais muito similares?

5. Custo Computacional e Escalabilidade

A pergunta central

Métodos iterativos custam $O(n^2)$ por iteração. Se precisam de k iterações, o custo total é $O(kn^2)$. Para k fixo, isso pode ser muito melhor que $O(n^3)$ dos métodos diretos. Mas k é fixo? Esta parte investiga.

```

1 import numpy as np
2 import time
3 from jacobi_seidel import jacobi, gauss_seidel
4 from sor import sor, omega_young
5 from gradiente_conjugado import cg
6
7 def gerar_tridiagonal(n, diag=4, sub=-1):
8     """Gera matriz tridiagonal n x n."""
9     A = diag * np.eye(n)
10    A += np.diag([sub] * (n-1), k=1)
11    A += np.diag([sub] * (n-1), k=-1)
12    b = np.ones(n)
13    return A, b
14
15 def medir_tempo(func, *args, repeticoes=3, **kwargs):
16     """Retorna o menor tempo entre 'repeticoes' execucoes."""
17     tempos = []
18     for _ in range(repeticoes):
19         t0 = time.perf_counter()
20         func(*args, **kwargs)
21         tempos.append(time.perf_counter() - t0)
22     return min(tempos)

```

Listing 5: custo.py — medição de tempo e escalabilidade

5.1. Como k varia com n ?

Q5.1. Para sistemas tridiagonais com $n \in \{10, 30, 50, 100, 200, 500\}$, execute Gauss-Jacobi, Gauss-Seidel e SOR(ω_{opt}) com tolerância 10^{-8} . Registre o número de iterações para cada n .

- Plote “número de iterações vs. n ” para os três métodos. A escala linear ou log-log revela melhor o padrão?
- Para Gauss-Jacobi e Gauss-Seidel, o número de iterações cresce como $O(n)$, $O(n^2)$ ou outra lei? Ajuste uma lei de potência $k \approx c \cdot n^\alpha$ com `np.polyfit` no espaço log-log.
- Para SOR ótimo, o crescimento é diferente? Isso é consistente com a previsão teórica $k_{SOR} = O(n)$ para a equação de Poisson?
- Combine custo por iteração com número de iterações para estimar o custo total de cada método em função de n . Compare com $O(n^3)$ (LU). A partir de qual n cada método iterativo supera o LU?

5.2. Tempo real de execução

Q5.2. Para $n \in \{50, 100, 200, 500\}$, use `medir_tempo` para registrar o tempo de execução de cada método. Plote tempo vs. n em escala log-log.

- (a) O expoente empírico ($T \propto n^\alpha$) é compatível com $O(kn^2)$, sendo $k = O(n^2)$ para Jacobi/GS e $k = O(n)$ para SOR?
- (b) Compare com `numpy.linalg.solve` (método direto). Plote na mesma figura. Qual método é mais rápido para cada n ?
- (c) Para $n = 500$, compare também com CG. Por que o CG é competitivo mesmo com custo $O(n^2)$ por iteração?

6. Comparação Global: Quando Usar Cada Método?

Síntese da investigação

Esta seção não fornece código novo: usa tudo que foi construído. O objetivo é desenvolver **julgamento numérico** — a capacidade de escolher o método certo para cada problema.

6.1. Análise comparativa estruturada

Q6.1. Preencha a tabela de síntese com base nos experimentos realizados nas seções anteriores. Onde a tabela pede “lei de crescimento”, use os ajustes de potência obtidos na Seção 5.

Aspecto	Gauss-Jacobi	Gauss-Seidel	SOR ótimo	Grad. Conj.
Req. sobre A				
Custo/iter. (flops)				
Iter. para Poisson 1D				
Paralelizável?				
Mem. necessária				
Impacto de $\kappa(A)$				
Melhor cenário				
Pior cenário				

Q6.2. Três engenheiros propõem solvers para o sistema de equações normais $A^T A x = A^T b$ de um problema de regressão com $n = 800$. $A^T A$ é SPD com $\kappa \approx 2000$.

- **Engenheiro 1:** Gauss-Jacobi com tolerância 10^{-10} .
- **Engenheiro 2:** SOR com $\omega = 1,5$ (sem calcular ω_{opt}).
- **Engenheiro 3:** CG com pré-condicionamento Jacobi.

Com base nos experimentos das seções anteriores:

- Qual solução você recomenda? Justifique quantitativamente.
- Quais são os riscos de cada abordagem?
- Há alguma informação adicional que você precisaria saber antes de decidir com certeza?

7. Projeto Integrador: Equação de Calor Estacionária

Contexto. A equação do calor estacionária em 1D é:

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0.$$

A discretização por diferenças finitas com n pontos interiores ($h = 1/(n+1)$) produz o sistema tridiagonal:

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i, \quad i = 1, \dots, n,$$

com $f(x) = \pi^2 \sin(\pi x)$ e **solução exata** $u(x) = \sin(\pi x)$. Este é exatamente o tipo de sistema para o qual os métodos iterativos foram desenvolvidos.

Q7.1. Para $n = 50$, monte o sistema tridiagonal e resolva-o com Gauss-Jacobi, Gauss-Seidel, SOR(ω_{opt}) e CG.

- Plote a solução numérica de cada método sobre a solução exata. São visualmente indistinguíveis?
- Compare os erros $\|u_{num} - u_{exata}\|_\infty$ para os quatro métodos. Há diferença de precisão entre eles?
- Preencha a tabela:

Método	Iterações	$\ u_{num} - u_{exata}\ _\infty$	Tempo (ms)	ρ
Gauss-Jacobi				
Gauss-Seidel				
SOR (ω_{opt})				
CG				

Q7.2. Repita o experimento para $n \in \{20, 50, 100, 200, 500\}$.

- Plote “número de iterações vs. n ” para os quatro métodos em escala log-log. Qual método apresenta crescimento mais lento?
- O erro $\|u_{num} - u_{exata}\|_\infty$ diminui com n ? Como? Este é o erro de *discretização* (do método de diferenças finitas), independente do solver iterativo.
- A partir de $n = 200$, qual método é claramente mais eficiente? Justifique com dados quantitativos.

Q7.3. Investigue o efeito do ponto de partida $x^{(0)}$ na convergência do CG. Para $n = 100$:

- Use $x^{(0)} = \mathbf{0}$ (padrão).
- Use $x^{(0)} =$ solução exata perturbada com ruído de magnitude 10^{-2} .
- Use $x^{(0)} =$ solução de $n = 50$ interpolada para $n = 100$ (“aquecimento a quente”).

Compare o número de iterações. O CG é sensível ao ponto inicial? Explique.

8. Desafio (Opcional — Pontuação Extra)

Desafio Computacional

As questões desta seção são opcionais e destinadas a alunos que desejam aprofundar a investigação além do exigido.

Q8.1. Critério de parada e qualidade da solução. O critério relativo $d_r^{(k)} = \|x^{(k)} - x^{(k-1)}\|_\infty / \|x^{(k)}\|_\infty < \varepsilon$ nem sempre garante que $x^{(k)}$ está próximo de x^* .

- (a) Construa um sistema 4×4 bem-condicionado e um mal-condicionado ($\kappa \approx 10^6$). Para cada um, execute Gauss-Seidel e registre o verdadeiro erro $\|x^{(k)} - x^*\|$ e o critério $d_r^{(k)}$ a cada iteração.
- (b) Em qual caso $d_r^{(k)}$ é um indicador confiável do erro real?
- (c) Proponha um critério de parada baseado no resíduo $\|Ax^{(k)} - b\|$ que seja mais robusto para sistemas mal-condicionados.

Q8.2. ILU como pré-condicionador do CG. O pré-condicionamento por fatoração LU Incompleta (ILU) mantém os zeros de A e aproxima $A \approx \tilde{L}\tilde{U}$.

- (a) Use `scipy.sparse.linalg.spilu` para construir um pré-condicionador ILU para a matriz Laplaciana 2D ($n \times n$, $n = 30$, gerada com `scipy.sparse.diags`).
- (b) Compare CG sem pré-cond., PCG+Jacobi e PCG+ILU em termos de iterações e tempo.
- (c) Calcule $\kappa(M^{-1}A)$ para o ILU e compare com os demais. O ILU justifica seu custo adicional de construção?

Q8.3. Jacobi paralelo vs. Gauss-Seidel sequencial. Jacobi pode ser paralelizado naturalmente; Gauss-Seidel, não.

- (a) Implemente uma versão vetorizada de Jacobi (sem laço explícito em i) usando operações NumPy. Meça o speedup em relação à versão com laço para $n = 1000$.
- (b) Simule 4 “processadores” dividindo as componentes de x em 4 blocos. Cada bloco atualiza suas componentes em paralelo (use um único vetor de leitura, quatro de escrita). Compare com Jacobi sequencial.
- (c) Discuta: para qual n a paralelização de Jacobi superaria Gauss-Seidel sequencial, dado que GS precisa de $\approx$ metade das iterações?

Entrega e Critérios de Avaliação

O que entregar

Formato de entrega (via Moodle, antes do prazo):

1. **Relatório PDF** com todas as respostas, tabelas preenchidas, gráficos gerados e análises escritas. Gráficos devem ter título, eixos rotulados e legenda.
2. **Arquivos Python** organizados: `jacobi_seidel.py`, `convergencia.py`, `sor.py`, `gradiente_conjugado.py`, `custo.py` e `main.py` (script que reproduz todos os resultados do relatório).
3. Cada arquivo deve conter uma **docstring** descrevendo o que faz.

Seção	Questões	Valor
1. Jacobi e Gauss-Seidel	Q1.1–Q1.4	2,0 pts
2. Critérios de convergência	Q2.1–Q2.3	1,5 pts
3. Método SOR	Q3.1–Q3.4	2,0 pts
4. Gradiente Conjugado	Q4.1–Q4.3	2,0 pts
5. Custo computacional	Q5.1–Q5.2	1,0 pts
6. Comparação global	Q6.1–Q6.2	1,0 pts
7. Projeto integrador	Q7.1–Q7.3	1,5 pts
Total obrigatório		11,0 pts
8. Desafio (extra)	Q8.1–Q8.3	até 1,5 pts

Critérios de qualidade:

- Código limpo, comentado e executável sem erros;
- Análises escritas com rigor: não basta dizer “convergiu mais rápido”, é preciso **quantificar** (número de iterações, razão de raios espectrais, expoente de escala) e **justificar** com a teoria;
- Gráficos com qualidade (títulos, legendas, escala adequada);
- Conexão entre resultados empíricos e teoria apresentada em aula;
- Discussão das **diferenças** entre os métodos — o ponto central desta investigação.

A. Esqueleto do Script Principal

```

1  """
2  main.py
3  Plano de Investigacao      Metodos Iterativos
4  Disciplina: Calculo Numerico
5  """
6  import numpy as np
7  import time
8  import matplotlib.pyplot as plt
9  from jacobi_seidel         import jacobi, gauss_seidel,
    raio_espectral
10 from convergencia          import criterio_linhas,
    criterio_sassenfeld, diagnostico
11 from sor                   import sor, omega_young,
    varredura_omega
12 from gradiente_conjugado   import cg, pcg
13 from custo                 import gerar_tridiagonal, medir_tempo
14
15 np.random.seed(42)
16
17 #          Secao 1: verificacao basica

18 A = np.array([[ 8, -1,  1],
19               [ 1,  6, -2],
20               [-1,  2,  7]], dtype=float)
21 b = np.array([14, 10, -5], dtype=float)
22
23 x_j, it_j, hist_j = jacobi(A, b, tol=1e-6)
24 x_gs, it_gs, hist_gs = gauss_seidel(A, b, tol=1e-6)
25 print(f"Jacobi:      {it_j} iteracoes | residuo = {hist_j
    [-1]:.2e}")
26 print(f"Gauss-Seidel: {it_gs} iteracoes | residuo = {hist_gs
    [-1]:.2e}")
27
28 #          TODO: Secao 2          diagnostico de convergencia

29 from convergencia import diagnostico
30 # diagnostico(A)
31
32 #          TODO: Secao 3          varredura de omega

33 # omegas = np.linspace(0.3, 1.95, 35)
34 # iters = varredura_omega(A, b, omegas)
35 # plt.plot(omegas, iters); plt.xlabel("omega"); plt.ylabel("
    iteracoes")
36
37 #          TODO: Secao 4          CG vs. SOR

```

```
38 # A20, b20 = gerar_tridiagonal(20)
39 # x_cg, it_cg, hist_cg = cg(A20, b20, tol=1e-8)
40
41 #          TODO: Secao 5          escalabilidade
42
43
44 #          TODO: Secao 7          equacao do calor
45
46
47 # n = 50; h = 1/(n+1)
48 # x_pts = np.linspace(h, 1-h, n)
49 # u_exata = np.sin(np.pi * x_pts)
50
51 plt.tight_layout()
52 plt.savefig("resultados_iterativos.pdf", dpi=150)
53 plt.show()
```

Listing 6: main.py — esqueleto para o relatório

B. Referências Bibliográficas

- [1] Saad, Y. *Iterative Methods for Sparse Linear Systems*. 2.^a ed. SIAM, 2003.
https://www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf
- [2] Golub, G. H.; Van Loan, C. F. *Matrix Computations*. 4.^a ed. JHU Press, 2013.
Cap. 10.
- [3] Burden, R. L.; Faires, J. D. *Numerical Analysis*. 10.^a ed. Cengage, 2015. Cap. 7.
- [4] Barrett, R. et al. *Templates for the Solution of Linear Systems*. SIAM, 1994.
<https://www.netlib.org/templates/>
- [5] SciPy Sparse — <https://docs.scipy.org/doc/scipy/reference/sparse.linalg.html>